```
            self->ts[probefunc] = timestamp;
}
pid$1:::return
/self->ts[probefunc]/
{
            @func_time[probefunc] = sum(timestamp \
                        - self->ts[probefunc]);
            self->ts[probefunc] = 0;
}
```

5.2.9 Collecting Information About an Application

As an alternative to specifying a process id number to be traced, you can specify a command to be traced. When you specify a command to be traced, DTrace collects data on that command process and all of the child processes of that command. DTrace displays the collected data after the specified command exits. To collect data about a command, use the `-c` option to specify the command on the `dtrace` command line or when you invoke your D script. If you use a D script, use the `DTrace $target` macro inside the script to capture the process id of the argument to the -c option on the command line.

The following script counts the number of times libc functions are called from a specified command:

```
#!/usr/sbin/dtrace -s
pid$target:libc::entry
{
            @[probefunc]=count();
}
Use the -c option to specify the target command:
# ./myDscript.d -c "man dtrace"
```

In this example, the first output you see is the dtrace man page. When you press the q key, you see output such as the following. Most of the output is omitted in this example.

```
dtrace: script ’./myDscript.d’ matched 2914 probes
dtrace: pid 1111 has exited
  ___lwp_private              1
  strncpy                   125
  strlen                   7740
```